

Atividade 5: Multiplexação de Entrada/Saída

**** *Data de entrega: 04 de Novembro de 2025*

*** *Este trabalho deve ser realizado utilizando a linguagem C. Trabalhos que utilizarem bibliotecas externas poderão receber nota zero na implementação, pois todos os trabalhos serão compilados utilizando o padrão destas linguagens. Isto vale para quem utiliza Windows, certifique-se de compilar e executar em máquinas com Linux/Unix antes de entregar a tarefa.*

*** *Lembre-se de justificar e comprovar suas respostas no relatório.*

*** *Relatório arquivo deverá ser em formato PDF*

*** *Detalhes da implementação:*

O código não pode apresentar nenhum warning quando compilado. Para cada warning exibido na compilação será descontado 10% da nota deste trabalho. (Certifique-se de compilar seu código com a flag `-Wall` do `gcc`). Adicione todos os comentários necessários no código para indicar o que foi feito para realizar cada uma das questões e identificar as diferentes funções. A ordem do seu código será levada em consideração.

*** *Entregáveis: relatório e códigos. Regra para atribuição de nota: Relatório 50% e Código 50%.*

Exercício

Este exercício tem como finalidade investigar o uso de multiplexação de entrada de saída utilizando as funções `select()` e `poll()`, as configurações do kernel do sistema operacional e o impacto direto no desempenho e na capacidade de um servidor em lidar com múltiplas conexões simultâneas.

1. Antes de modificar o código, investigue e descreva com suas próprias palavras o funcionamento do processo de multiplexação de entrada e saída em um Kernel Linux. Sua explicação deve responder às seguintes questões:
 - Qual a diferença entre uma operação de E/S bloqueante, não bloqueante e assíncrona?
 - Como esses conceitos podem ser aplicados em sistemas de redes de computadores?
 - Existem exemplos de aplicações que utilizam multiplexação de E/S no cenário de redes de computadores? Quais?
2. Modifique o código de um servidor TCP concorrente (baseado em `fork()`) para que ele seja capaz de atender as requisições utilizando um único processo:
 - Agora implemente uma nova versão do servidor utilizando a função `select()` para tratar múltiplos sockets em um único processo;
 - Qual a diferença entre ambas as implementações? Utilize prints para comprovar seus argumentos.
 - Agora implemente a multiplexação de entradas e saídas utilizando a função `poll()`. Quais as diferenças entre as funções `poll()` e `select()`? Ambas possuem o mesmo comportamento do servidor concorrente? Comprove utilizando prints e informações do sistema

3. Desenvolva um script (pode usar `bash` com `&` ou `xargs -P`) para automatizar os testes de carga no seu servidor. O objetivo é comparar o tempo de resposta entre o servidor concorrente e o multiplexado.

Execute o servidor com `TEMPO_SLEEP` definido. Em seu script, varie o valor de `NÚMERO DE CLIENTES` de 10 a 100, com um passo de 10.

- O número de clientes que conseguem estabelecer a conexão imediatamente (estado `ESTABLISHED`), pode ser verificado com o comando `ss` ou `netstat`.
 - Anote os resultados em uma tabela: número de clientes, conexões imediatas, tempo médio de resposta.
4. Agora, desenvolva um servidor que seja capaz de tratar conexões TCPs e mensagens UDP, utilizando `select()`.
 - Utilize a implementação do servidor concorrente com tratamento de zumbis como base
 - Modifique o código do cliente para utilizar uma conexão TCP ou enviar pacotes UDP, utilize um novo argumento posicional que indique o tipo de protocolo de transporte (0 para UDP e 1 para TCP).
 - Conecte clientes UDP e TCP no servidor e utilize ferramentas do sistema operacional (como `netstat` e `ss`) para mostrar o servidor tratando requisições de ambos os tipos de clientes.

Dicas e observações finais:

- Consulte os slides da disciplina e os capítulos 3 e 6 do livro texto guia para resolver os pontos anteriores.
- Use máquinas diferentes (servidor da VM netlabs e o cliente pode ser o host) se quiser simular latência de rede real; em `localhost` o comportamento tende a ser mais rápido e pode mascarar efeitos de fila por aceitação rápido. Indique claramente no relatório as máquinas utilizadas e sua estratégia.
- Para garantir simultaneidade maior, use `xargs -P` ou `parallel` para disparar processos em paralelo.
- Sempre documente os comandos exatos usados e cole as saídas no relatório (`stdout/stderr`).

O relatório deve conter:

- Código modificado do servidor (com as mudanças destacadas).
- Scripts/estratégias usadas para lançar clientes simultâneos.
- Resultados experimentais (tabela e observações).
- Conclusões sobre a multiplexação, as diferenças com o servidor concorrente e as vantagens e desvantagens de ambas as implementações